AURA AI OS

Complete Setup, 24x7 Paper Service and Operations Guide

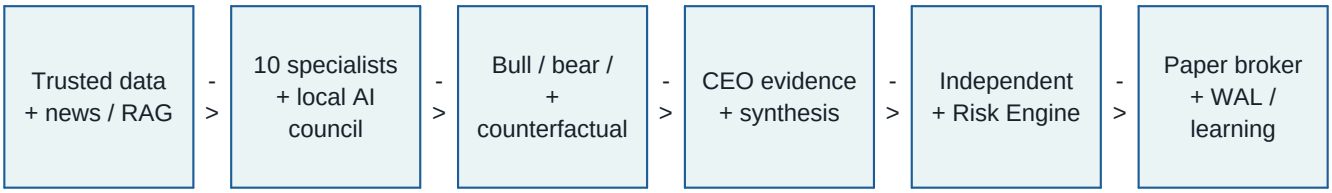
Validated integrated release candidate for multi-agent intelligence, local AI, trusted knowledge, autonomous research, forward shadow learning and risk-first paper/demo operation.

RELEASE CLASS	**PAPER / DEMO RESEARCH CANDIDATE**
VALIDATION	**398 LOCAL TESTS PASS; CI GATED**
LIVE MONEY	**DISABLED BY DEFAULT**
PRIMARY START	**START_AURA_OLLAMA.cmd**

Operator guide - generated from the validated AURA repository

1. What is included

AURA is not a single indicator bot. The integrated codebase separates intelligence from authority and uses a deterministic, auditable path for every decision.



Capability	Status	Evidence / note
Multi-market scanner	IMPLEMENTED	Closed-candle, multi-timeframe runtime foundations.
10 specialist roles	IMPLEMENTED	HTF, SMC/ICT, technical, volume, forecast, options, macro, cross-market, regime, execution.
Multi-model AI	IMPLEMENTED	Ollama council with bounded concurrency and structured evidence.
CEO + debate	IMPLEMENTED	Bull, bear and counterfactual review before deterministic synthesis.
Knowledge and news	IMPLEMENTED	Knowledge firewall, local authorized corpus, official feeds and GDELT.
Self-learning research	IMPLEMENTED	Shadow outcomes, missed opportunities, evolution and governed promotion.
Risk and accounting	IMPLEMENTED	Independent risk veto, paper broker, portfolio ledger, WAL and recovery.
Local voice	IMPLEMENTED	OS-native spoken alerts; no cloud key required.
Graphical dashboard	PARTIAL	Typed command center and status JSON exist; full GUI remains.
Telegram / WhatsApp	PENDING	External delivery adapter and verified recipient are required.

Safety boundary: AI may advise, explain and propose research. It cannot calculate financial quantities by guessing, bypass RiskEngine, approve itself, or enable live money.

2. Choose the correct deployment

Capability	Status	Evidence / note
Windows one-click	READY	Best for local voice, Ollama and future MT5 demo use.
Windows background task	READY	Starts at user logon and restarts after failures.
Docker Compose	READY	Cross-platform 24x7 public-data service; voice is off inside container.
Linux systemd user service	READY	24x7 VPS/laptop paper research with persistent runtime state.
Dhan paper	ACCOUNT GATE	Requires user-owned Dhan credentials and eligible data access.
MT5 demo	ACCOUNT GATE	Requires Windows MT5 terminal and verified DEMO account.
Angel One	PENDING	Concrete SmartAPI adapter is not yet in this release.
Live money	BLOCKED	Requires Phase 15, broker-origin forward evidence and human approval.

Recommended demonstration path

- Use Windows one-click when showing local Multi-AI plus voice.
- Use Docker Compose or Linux systemd for unattended public-data shadow training.
- Use Dhan or MT5 only after the public stack is healthy and credentials stay outside Git.

No setup mode in this guide sends live-money orders. Public autonomy sends no broker orders. Dhan self-evolution uses internal paper execution. MT5 must remain DEMO.

3. Windows one-click setup

Prerequisites

- Windows 10 or 11, 64-bit.
- Python 3.11 or 3.12 with Add Python to PATH enabled.
- Git, at least 12 GB free disk space, and stable internet.
- Ollama installed and running. A 16 GB RAM system should start with one smaller model.

Install local AI models

```
ollama pull qwen3
ollama pull deepseek-r1
```

If hardware is limited, install only one model. Model names must exactly match `ollama list`.

Clone and launch

```
git clone https://github.com/svk1dec2018-ai/AURA-AI-OS-ya-ai-trading-system.git
cd AURA-AI-OS-ya-ai-trading-system
git switch agent/client-paper-release-candidate
START_AURA_OLLAMA.cmd
```

- The first run creates `.venv`, installs dependencies and runs paper preflight.
- It starts public data, history, news, the Multi-AI council and shadow strategy lab.
- Voice announces shadow signals and states that no order was sent.

Custom PowerShell launch

```
powershell -ExecutionPolicy Bypass -File scripts/start_aura_ollama.ps1 `
  -Models qwen3,deepseek-r1 -Provider coinbase -Timeframe 5s
```

4. Windows background service-like mode

Windows Task Scheduler starts at logon, restarts after failure and avoids turning AURA into an unsafe privileged Windows service.

Step 1 - complete one foreground run

```
START_AURA_OLLAMA.cmd
```

Step 2 - install and start the task

```
powershell -ExecutionPolicy Bypass -File `
  scripts/install_aura_windows_task.ps1 -StartNow
```

Operations

```
Get-ScheduledTask -TaskName 'AURA AI OS Paper Research'
Start-ScheduledTask -TaskName 'AURA AI OS Paper Research'
Stop-ScheduledTask -TaskName 'AURA AI OS Paper Research'
Unregister-ScheduledTask -TaskName 'AURA AI OS Paper Research'
```

- Keep Ollama configured to start with Windows.

- The background task disables voice to avoid speech from a hidden session.

- Research state persists in `runtime/free_public_autonomy`. Back up this folder.

The task runs with the current user at limited privilege. It receives no live approval variable or broker credential from the installer.

5. Docker Compose 24x7 setup

Start Ollama on the host, then build AURA

```
docker compose -f compose.paper.yml up -d --build
docker compose -f compose.paper.yml ps
docker compose -f compose.paper.yml logs -f
```

Optional `.env` values

```
AURA_PUBLIC_PROVIDER=coinbase
AURA_DECISION_TIMEFRAME=5s
AURA_OLLAMA_URL=http://host.docker.internal:11434
AURA_OLLAMA_MODELS=qwen3,deepseek-r1
```

- Runtime state is stored in the named volume `aura-paper-state`.

- The container filesystem is read-only except `/tmp` and the runtime volume.

- Linux capabilities are dropped and `no-new-privileges` is enabled.

- Live approval variables are explicitly empty inside the service.

Stop, restart and back up

```
docker compose -f compose.paper.yml restart
docker compose -f compose.paper.yml down
docker run --rm -v aura-paper_aura-paper-state:/state `
  -v ${PWD}:/backup alpine tar czf /backup/aura-state.tgz -C /state .
```

Do not use `down -v` unless you intentionally want to delete paper learning state.

6. Linux or VPS systemd user service

Prepare AURA

```
python3.12 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
pip install -e '.[dev]'
python examples/run_production_preflight.py --mode paper --connector public
```

Install the user service

```
chmod +x scripts/install_aura_user_service.sh
./scripts/install_aura_user_service.sh
systemctl --user status aura-paper.service
journalctl --user -u aura-paper.service -f
```

Keep it running after logout

```
sudo loginctl enable-linger $USER
```

- The installer runs paper preflight before creating the service.

- The unit restricts writes to AURA's runtime directory.

- Ollama may run as a separate host service at `127.0.0.1:11434`.

Service operations

```
systemctl --user restart aura-paper.service
systemctl --user stop aura-paper.service
systemctl --user disable aura-paper.service
```

7. Dhan Indian-market paper setup

Dhan provides Indian-market live data for internal paper learning. Credentials must belong to the operator and must never be pasted into source code or GitHub.

Set credentials for the current PowerShell session

```
$env:AURA_DHAN_CLIENT_ID='YOUR_CLIENT_ID'
$env:AURA_DHAN_ACCESS_TOKEN='YOUR_ACCESS_TOKEN'
python examples/run_production_preflight.py --mode paper --connector dhan
python examples/check_dhan_universe.py
python examples/run_dhan_self_evolving_paper.py
```

- Confirm the instrument master, data subscription and option access.
- The runtime uses broad radar selection before expensive deep analysis.
- Orders remain internal paper orders; do not reinterpret them as broker fills.

Angel One SmartAPI is not silently substituted for Dhan. A separate official adapter, static-IP/rate-rule review and account validation are still required.

8. MT5 forex and metals demo setup

Prerequisites

- Windows MetaTrader 5 installed and logged into a DEMO account.
- Python package `MetaTrader5` installed in AURA's `.venv`.
- Market Watch symbols enabled for XAUUSD and required instruments.

Configure only a DEMO account

```
.venv\Scripts\python.exe -m pip install MetaTrader5
$env:AURA_MT5_DEMO_LOGIN='YOUR_DEMO_LOGIN'
$env:AURA_MT5_DEMO_PASSWORD='YOUR_DEMO_PASSWORD'
$env:AURA_MT5_DEMO_SERVER='YOUR_DEMO_SERVER'
python examples/run_production_preflight.py --mode demo --connector mt5_demo
python examples/run_mt5_self_evolving_paper.py
```

- AURA checks account mode before guarded MT5 demo broker calls.
- Symbol suffixes and contract sizes are broker-specific and must be verified.

Never place a live login in variables named `AURA_MT5_DEMO_*`. Account-mode mismatch must stop the runtime rather than bypass the demo guard.

9. Knowledge, news and authorized content

Included sources

- Official RBI and SEBI feeds where available.
- GDELT news context.
- Optional FRED and Alpha Vantage using user-owned free keys.
- Local authorized/public corpus with source, date, trust and hash metadata.

Optional environment variables

```
$env:AURA_FRED_API_KEY='YOUR_KEY'
$env:AURA_ALPHA_VANTAGE_API_KEY='YOUR_KEY'
$env:AURA_SEC_USER_AGENT='Your Name your@email.example'
```

Authorized document ingestion

- Use only owned, public-domain or properly licensed material.
- Record original source, publication date, trust score and content hash.
- Do not scrape copyrighted books, paid courses or unauthorized transcripts.
- Knowledge is evidence only; it does not calculate quantity or place trades.

Point-in-time policy: information observed after a decision timestamp is rejected from that historical decision, even if its publication date appears earlier.

10. Daily operations and health checks

Before startup

- Run production preflight for the selected paper/demo connector.
- Confirm live approval variables are empty.
- Confirm system clock, disk space, network and runtime permissions.
- Confirm Ollama responds and selected models appear in `ollama list`.

During operation

- Watch status JSON, structured logs, AI timeouts and data-quality counters.
- Treat NO-TRADE as a valid decision, not a runtime failure.
- Investigate reconciliation, stale-feed, future-data and operational incidents.

Validated maintenance commands

```
ruff check .
pytest -q
python -m compileall -q aura
python -m aura.ops.repository_audit --check
python -m build
```

Important runtime paths

Capability	Status	Evidence / note
Public autonomy	STATE	runtime/free_public_autonomy
Dhan paper	STATE	runtime/dhan_self_evolving_paper
MT5 paper	STATE	runtime/mt5_self_evolving_paper
Governance evidence	AUDIT	artifacts/governance
Client guide	DOC	output/pdf/AURA_SETUP_AND_OPERATIONS.pdf

11. Troubleshooting

Capability	Status	Evidence / note
Ollama unreachable	CHECK	Open Ollama, run `ollama serve`, verify `/api/tags` and firewall.
Model not found	CHECK	Use exact name from `ollama list`; pull it once before service start.
Python import error	CHECK	Activate `.venv` and install from repo root.
No AI decisions	CHECK	Wait for minimum history and inspect data and timeout counters.
No signals	NORMAL	NO-TRADE can be correct; inspect evidence before changing thresholds.
Windows task stops	CHECK	Run launcher in foreground for the exact error.
systemd restart loop	CHECK	Use `journalctl --user -u aura-paper.service -n 200`.
Docker cannot reach Ollama	CHECK	Verify host address, Ollama bind policy and firewall.
Dhan unauthorized	STOP	Use official refresh flow; never copy another person's token.
MT5 mismatch	STOP	Use a verified DEMO account; do not bypass the demo guard.

Recovery rule

- Stop the service cleanly and back up runtime state and logs.
- Run audit, tests and preflight.
- Restart in public paper mode and confirm recovery/reconciliation state.
- Never fix a failure by enabling live authority or disabling RiskEngine.

12. Security and final acceptance

Never commit

- Broker usernames, passwords, tokens, TOTP seeds or sessions.
- `.env` files containing real values.
- Live approval identifiers or risk acknowledgements.
- Licensed data or copyrighted material without redistribution rights.

Client acceptance checklist

- GitHub checks and CodeQL are green.
- Paper preflight is READY and live-money-disabled passes.
- The deployment survives restart and retains state.
- Data-quality and AI failures are visible, not silently ignored.
- Every decision has structured evidence and an audit identity.
- No strategy can move to live without evidence and human approval.

Before unrestricted live production

- Credential-backed broker conformance and reconciliation validation.
- Venue-specific lot, tick, margin, expiry and settlement checks.
- Long-duration broker-origin forward evidence across regimes.
- All mandatory Phase 1-15 evidence gates.
- Human strategy approval and smallest-size controlled canary.

Final truth: this release is ready for client review, public-data autonomy, local Multi-AI demonstrations and governed paper/demo research. It is not a guaranteed-profit product and is not certified for unrestricted real-money execution.

Repository

github.com/svk1dec2018-ai/AURA-AI-OS-ya-ai-trading-system

Primary integrated review: pull request 25